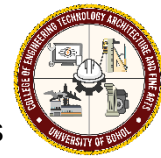




UNIVERSITY OF BOHOL
Tagbilaran City, Bohol, Philippines



COLLEGE OF ENGINEERING, TECHNOLOGY,
ARCHITECTURE AND FINE ARTS

BACHELOR OF SCIENCE IN COMPUTER ENGINEERING
CPE COGNATE/TRACK COURSE 1
CPEP 323 A

NEON DODGE: LEVEL OVERDRIVE

*Hand-Tracking Arcade Dodge Game — Technical
Datasheet*

Submitted by:

SUPREMO, EDREAN .

Table of Contents

Table of Contents	2
1. Executive Summary	4
2. Introduction	4
2.1 Project Goal	4
2.2 Target Audience	4
2.3 Platform	4
2.4 High-Level Gameplay Loop	4
3. How to Play (User Manual)	4
3.1 Controls and Gestures	4
3.2 Objectives	5
3.3 Win / Lose Conditions	5
3.4 Level Progression	5
3.5 UI Elements and On-Screen Indicators	5
3.6 Tips	5
4. System Workflow	6
4.1 Runtime Pipeline	6
4.2 Frame-Rate and Timing	6
4.3 Levels.py Parameter Mapping	6
5. Libraries Used	7
5.1 OpenCV (cv2)	7
5.2 MediaPipe (mediapipe)	7
5.3 pygame	8
5.4 NumPy (numpy)	8
5.5 threading	8
6. Hand Tracking Implementation	8
6.1 Frame Capture (OpenCV)	8
6.2 MediaPipe Integration	8
6.3 Landmark-to-Game-Space Conversion	9
6.4 Smoothing	9
6.5 Gesture Recognition	9
6.6 Tracking Robustness	9
7. Game Mechanics, Effects, and Functionality	9
7.1 Entity Overview	9
7.2 Exploder Behaviour	10
7.3 Laser Behaviour	10
7.4 SunOrb and BigOrb Behaviour	10

7.5 Black Hole (Level 3)	10
7.6 Power-Up System.....	11
8. Audio: Music and Sound Effects	11
8.1 Audio Assets	11
8.2 Audio Triggering and Mixing	11
8.3 Music Flow	12
9. Visual and Audio Implementation Details.....	12
9.1 Rendering Pipeline	12
9.2 Transparency and Glow Effects	12
9.3 Particle Systems	12
9.4 Screen Effects.....	12
10. Code Snippets and Pseudocode (Appendix)	12
10.1 OpenCV → MediaPipe Frame Loop	13
10.2 Smoothing and Game-Space Conversion.....	13
10.3 Level Configuration to Spawn Logic	13
10.4 Full levels.py Content.....	13
11. Testing, Performance, and Limitations	14
11.1 Recommended Camera Settings	14
11.2 Performance Targets.....	14
11.3 Known Edge Cases	14
11.4 QA Checklist	14
12. References and Assets.....	16
12.1 Source Files.....	16
12.2 Asset Files.....	16
12.3 Third-Party Libraries.....	16
12.4 Credits	16
13. One-Page Summary	17

1. Executive Summary

Neon Dodge: Level Overdrive is a real-time arcade-style survival game in which the player controls a neon avatar using bare-hand gestures captured by a standard webcam. Using OpenCV for frame capture and MediaPipe for hand landmark detection, the game translates the position of the player's index finger tip into smooth on-screen movement. Across three escalating levels — The Awakening, Pulse Overdrive, and Chaos Protocol — the player must survive 60 seconds per stage while dodging exploders, laser bolts, sun orbs, and, in the final stage, a massive black hole. Power-ups (shield, slow-motion, bonus HP) spawn periodically to give the player tactical relief. The project demonstrates real-time computer-vision integration, a modular entity system, layered audio management, and cinematic screen effects, all running at a locked 60 FPS inside a pygame fullscreen renderer.

2. Introduction

2.1 Project Goal

Neon Dodge: Level Overdrive was created to demonstrate the viability of touchless, computer-vision-based game control as an accessible and engaging alternative to keyboard or gamepad input. The primary technical goal was to achieve responsive, low-latency hand tracking using commodity hardware — specifically any webcam supported by OpenCV — while maintaining stable 60 FPS gameplay and cinematic visual polish.

2.2 Target Audience

The game targets casual players aged 13 and above who are comfortable with webcam-based interaction. It is also designed as an educational project for students and developers interested in integrating MediaPipe and OpenCV into real-time interactive applications.

2.3 Platform

The game runs on any system capable of running Python 3.10+ with pygame, OpenCV, and MediaPipe installed. It targets Windows, macOS, and Linux. The renderer detects the monitor's native resolution at launch and scales the 900x600 virtual canvas to fill the screen via letterboxing.

2.4 High-Level Gameplay Loop

Each session follows a fixed progression: a photosensitivity warning screen, background music, a level intro card, 60 seconds of real-time dodge gameplay, and — on success — a cinematic stage-cleared transition. On player death the level restarts. After completing all three levels, a scrolling credits screen is displayed. The player's sole interaction with the game outside menus is moving their index finger in front of the camera.

3. How to Play (User Manual)

3.1 Controls and Gestures

There are no buttons or keyboard commands required during active gameplay. All movement is gesture-driven:

- Point your index finger upward so that the fingertip (landmark 8) is above its proximal joint (landmark 6). The game detects this as the 'pointing' state and enables movement.
- Move your pointing hand across the camera's field of view. The player avatar follows the smoothed index fingertip position.
- On menu / transition screens, raise all four fingers to trigger 'Continue' (gesture_continue), or raise only the index and middle fingers for 'Quit' (gesture_quit).
- The camera preview (150x110 px) displayed in the bottom-left corner shows the live feed with green skeleton overlay so you can calibrate your hand position.

3.2 Objectives

Survive for 60 seconds per level without losing all three HP points. The progress bar at the top of the screen fills left-to-right as time elapses. A small orb tracks the current progress position.

3.3 Win / Lose Conditions

- Win: The level_timer countdown reaches zero with at least 1 HP remaining. A cinematic stage-cleared screen fires, followed by a countdown to the next level.
- Lose: The player's HP drops to zero. The screen fades to black over 90 frames (~1.5 s), then the level restarts automatically with a fresh intro card.

3.4 Level Progression

There are three levels. Completing Level 3 (Chaos Protocol) triggers the credits sequence and ends the game. At any time during gameplay, pressing keyboard keys 1, 2, or 3 jumps directly to the corresponding level for testing purposes. ESC quits immediately from any screen.

3.5 UI Elements and On-Screen Indicators

Element	Location	Description
Progress Bar	Top-centre	500px wide cyan bar; fills as survival time elapses
Survived Timer	Top-left	Shows elapsed seconds (e.g. 'SURVIVED: 42s')
HP Pips	Top-right	3 cyan circles; dimmed when HP is lost
Power-up HUD	Top-right / above player	Badge + timer bar showing active power-up type and % remaining
Camera Preview	Bottom-left	150x110 live webcam feed with hand skeleton overlay
Slow-mo Tint	Full screen	Blue pulse tint when slow-motion power-up is active

3.6 Tips

- Stay in the centre-left third of the camera — the edge-expansion mapping means you do not need to move your hand to the extreme edges to reach screen borders.
- During the laser warning (dashed grey line), move perpendicular to the line's direction to clear the bolt's path.
- The shield power-up absorbs the next hit entirely — save it for dense hazard clusters in Level 3.
- In Level 3, move away from centre when the black hole appears; gravity is weaker when you move away from it.

4. System Workflow

4.1 Runtime Pipeline

The game runs a single main thread for rendering and game logic, with hand tracking executing on a dedicated background thread (`HandTracker._update_loop`) to prevent camera I/O from blocking the render loop.

The runtime pipeline in order is:

1. Camera Input: `OpenCV VideoCapture(0)` reads a 640x480 frame at up to 60 FPS with a buffer size of 1 to minimise latency.
2. OpenCV Preprocessing: The frame is horizontally flipped (mirror effect) and converted from BGR to RGB.
3. MediaPipe Hand Detection: The RGB frame is passed to `mp.solutions.hands.Hands.process()`. Landmark coordinates are returned if a hand is detected.
4. Landmark Processing: Landmark index 8 (index fingertip) x/y coordinates are extracted. A 15%-margin edge-expansion mapping is applied with `np.interp`, and the result is stored as `_raw_pos`.
5. Gesture / Position Mapping: `index_pos` is lerped 55% toward `_raw_pos` each camera frame, smoothing jitter. Gesture flags (`gesture_continue`, `gesture_quit`, `is_pointing`) are derived from relative landmark heights.
6. Game Logic: In the main loop (60 FPS), `Player.update()` reads `tracker.get_move_vec()`, which returns a normalised direction vector toward `index_pos` if the player is pointing. Exploders, lasers, sun orbs, big orbs, and the black hole are updated. Collision checks run per entity.
7. Entity Spawning: Timers derived from `levels.py` parameters (`exploder_rate`, `laser_rate`, `sun_orb_rate`) control spawn cadence. Cooldown variables (`laser_cd`, `sun_orb_cd`) count down each frame.
8. Rendering and Audio: All entities draw onto `virtual_screen` (900x600). The virtual surface is scaled to the monitor resolution with `smoothscale`, offset by a random shake vector, and blitted to the physical display. Audio channels are dedicated per sound type.
9. Scoring and Persistence: Survival time is tracked via `level_timer` (decrements each game frame). HP is persisted across entity spawns within a level and reset on level restart or advancement.

4.2 Frame-Rate and Timing

All game logic runs at exactly 60 FPS enforced by `clock.tick(60)`. Spawn rates in `levels.py` are expressed in frames. The conversion formula is: $\text{seconds} = \text{frames} / 60$. For example, `exploder_rate = 240` means one spawn every 4 seconds. The camera capture is also requested at 60 FPS, but the actual delivery rate depends on hardware; the background thread runs as fast as the camera allows without blocking the main loop.

4.3 Levels.py Parameter Mapping

The following table shows how each parameter in `LEVELS[]` maps to observable runtime behaviour, using all three levels as examples:

Parameter	Level 1	Level 2	Level 3	Effect at Runtime
<code>exploder_rate</code>	240 (4 s)	140 (2.3 s)	120 (2 s)	Frames between Exploder spawns. Lower = more frequent.

exploder_count	2	3	5	How many Exploders spawn per trigger. Level 3 overrides to 5 in code.
laser_rate	120 (2 s)	60 (1 s)	30 (0.5 s)	Frames between laser spawns. Level 3 fires every half-second.
laser_speed	22 px/frame	34 px/frame	28 px/frame	Bolt travel speed. Level 2 is fastest; Level 3 trades speed for density.
laser_mode	single	rapid	single	'rapid' mode: Level 2 only queues one laser at a time (no stacking). 'single' allows stacking.
laser_grace	120 (2 s)	0	0	Frames at level start during which no lasers spawn. Level 1 gives 2 s grace.
mini_count	4	6	6	Fragments spawned per Exploder explosion.
particle_speed	3	6	7	Speed of MiniCircle fragments in px/frame.
sun_orb_rate	0	300 (5 s)	240 (4 s)	Frames between SunOrb pair spawns. 0 = disabled (Level 1).
has_big_orb	False	False	True	Enables the BigOrb hazard. Level 3 only.
big_orb_freq	0	0	999999	Not used directly; Level 3 manages BigOrb spawns manually every 300 frames.
music	music.mp3 3	music2.mp3	music3.mp3	Background music track loaded by play_level_music().

5. Libraries Used

5.1 OpenCV (cv2)

OpenCV handles all camera I/O and image processing. Key usage includes:

- `cv2.VideoCapture(0)` opens the default webcam with explicit `640x480@60FPS` and `BUFFER_SIZE=1` settings.
- `cv2.flip(frame, 1)` mirrors the frame horizontally so movement feels natural.
- `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` converts to RGB before passing to MediaPipe.
- `cv2.cvtColor(raw_frame, cv2.COLOR_BGR2RGB)` is called again in `draw_preview()` to convert back for pygame rendering.

Configuration: Detection confidence is set to 0.2 (permissive) to allow partial hand visibility near screen edges.

5.2 MediaPipe (mediapipe)

MediaPipe's Hands solution provides 21 3D hand landmarks in normalised coordinates [0.0, 1.0].

- `mp.solutions.hands.Hands(static_image_mode=False, model_complexity=0, max_num_hands=1, min_detection_confidence=0.2, min_tracking_confidence=0.2)` initialises the tracker. `model_complexity=0` uses the lightweight model for better performance.
- `mp.solutions.drawing_utils.draw_landmarks()` renders the green skeleton overlay onto the raw frame for the camera preview.
- Landmark index 8 (`INDEX_FINGER_TIP`) provides the cursor position. Index 6 (`INDEX_FINGER_PIP`) is used to detect the 'pointing' state (`tip.y < lm[6].y`).

- Landmarks 8, 12, 16, 20 (fingertips) are compared to their respective -2 landmarks to detect all-fingers-up (Continue) and two-fingers-up (Quit) gestures.

5.3 pygame

pygame is the primary game framework. Key subsystems used:

- `pygame.display`: Fullscreen surface creation, smoothscale letterboxing, caption, icon.
- `pygame.mixer`: Audio playback with 16 channels, 44100 Hz, 16-bit stereo. Channel 0 is reserved for music; channels 1-6 are dedicated per SFX type.
- `pygame.Surface` with `pygame.SRCALPHA`: Used extensively for all glow effects, transparency layers, and particle rendering to avoid opaque black boxes.
- `pygame.math.Vector2`: All position and velocity calculations use `Vector2` for clean 2D math including `.lerp()`, `.normalize()`, `.length()`, and `.distance_to()`.
- `pygame.time.Clock`: Locks the game to 60 FPS via `clock.tick(60)`.
- `pygame.font.SysFont('Courier New', ...)`: All text rendering uses Courier New for the monospace retro aesthetic.

5.4 NumPy (numpy)

NumPy's `np.interp()` is used in `tracking.py` for the edge-expansion coordinate mapping: `screen_x = np.interp(tip.x, [0.15, 0.85], [0, WIDTH])`. This remaps the 10%-90% camera band to the full game screen width, allowing the player to reach screen edges without moving their hand off-camera.

5.5 threading

Python's `threading.Thread` is used to run `HandTracker._update_loop()` as a daemon thread, fully decoupling camera capture from the 60 FPS game loop. Shared state (`index_pos`, `gesture flags`, `raw_frame`) is written by the background thread and read by the main thread without explicit locking (reads/writes of Python objects are GIL-protected for simple assignments).

6. Hand Tracking Implementation

6.1 Frame Capture (OpenCV)

The `HandTracker` constructor opens the webcam and sets capture properties before starting the background thread. The critical `BUFFER_SIZE=1` setting prevents frame lag by discarding buffered frames — without this, the game would display stale camera data with noticeable input delay.

```
self.cap = cv2.VideoCapture(0)
self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
self.cap.set(cv2.CAP_PROP_FPS, 60)
self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
```

Figure 1 — Camera initialisation with low-latency settings

6.2 MediaPipe Integration

Each iteration of the background loop reads a new frame, flips and converts it, and calls `hands.process()`. The result's `multi_hand_landmarks` list contains one entry per detected hand. Only the first entry is used (`max_num_hands=1`). The `mp_draw.draw_landmarks()` call

annotates the raw frame with a green skeleton that is later displayed in the in-game camera preview.

6.3 Landmark-to-Game-Space Conversion

MediaPipe returns normalised x/y coordinates in [0.0, 1.0]. The conversion to game-space pixels uses a margin-expanded linear interpolation:

```
margin = 0.15
screen_x = np.interp(tip.x, [margin, 1.0 - margin], [0, self.width])
screen_y = np.interp(tip.y, [margin, 1.0 - margin], [0, self.height])
```

Figure 2 — Edge-expansion mapping (*tracking.py, _update_loop*)

The margin value of 0.15 means that when the fingertip is at 15% of the camera width, the game cursor is at 0% (left edge). When at 85%, the cursor is at 100% (right edge). This lets players reach all four screen edges without their hand leaving the camera view.

6.4 Smoothing

Raw landmark coordinates contain per-frame jitter from camera noise and detection uncertainty. A simple exponential moving average (lerp) is applied each camera frame:

```
self._raw_pos = Vector2(screen_x, screen_y)
self.index_pos = self.index_pos.lerp(self._raw_pos, 0.55)
```

Figure 3 — 55% lerp smoothing applied each camera frame

A lerp factor of 0.55 was tuned empirically to eliminate visible jitter while maintaining perceived instantaneous response. A value of 1.0 would be unsmoothed; a value near 0.1 would feel sluggish.

6.5 Gesture Recognition

Three gesture states are computed each camera frame. `is_pointing` checks whether the index tip (landmark 8) is higher (smaller y) than the proximal joint (landmark 6), indicating an extended index finger. `gesture_continue` triggers when all four finger tips are above their -2 landmarks (open hand). `gesture_quit` triggers when only the index and middle fingers are extended.

6.6 Tracking Robustness

Setting `min_detection_confidence=0.2` and `min_tracking_confidence=0.2` allows MediaPipe to maintain tracking on partially occluded hands, which is common when the player moves their hand near screen edges or under varied lighting. When no hand is detected (`results.multi_hand_landmarks` is empty), the gesture flags are reset to False and `index_pos` retains its last known value. The player avatar therefore freezes in place rather than jumping, which is less disorienting than a snap to center.

7. Game Mechanics, Effects, and Functionality

7.1 Entity Overview

Entity	Class	Description	Levels Active
Player	Player	Cyan neon circle (radius 12). Moves via hand tracking. Has trail, glow, hit particles, and power-up auras.	1, 2, 3

Exploder	Exploder	90-frame countdown orb. Colour shifts from blue to red. Shakes on countdown. On death spawns MiniCircles.	1, 2, 3
MiniCircle	MiniCircle	Neon fragment ejected radially from Exploder death. Has a 10-point trail. Speed = particle_speed.	1, 2, 3
Laser	Laser	Fast projectile. Shows a 45-frame dashed warning line, then fires a level-coloured bolt with trail.	1, 2, 3
SunOrb	SunOrb	Bright white orb (radius up to 55). Grows over 1.7 s with screen shake and flame particles. Spawned in pairs.	2, 3
BigOrb	BigOrb	Larger version of SunOrb (radius up to 68). Spawned in groups of 3. Level 3 only.	3
BlackHole	BlackHole	Triggered in the final 30 s of Level 3. Grows to 220 px radius. Pulls all entities. Instakills on contact.	3
PowerUp	PowerUp	Floating collectable. Types: shield, slow, hp. Blinks before expiring.	1, 2, 3

7.2 Exploder Behaviour

An Exploder spawns at a random position (100-800px x, 100-500px y) with a 90-frame fuse timer. Each frame the timer decrements; the colour linearly shifts from blue (timer=90) to red (timer=0). Radius grows from 25 to ~40px. A shake_offset (proportional to elapsed fuse) makes the orb tremble. When the timer reaches zero, it is flagged dead, triggers a shake event (shake_amount += 18), plays the explosion sound, and spawns mini_count MiniCircle fragments at particle_speed.

7.3 Laser Behaviour

A Laser selects a random entry edge (top/bottom/left/right), positions its origin 30px off-screen, and fires straight across. The WARN_FRAMES constant (45 frames) provides a warning phase during which a dashed grey line shows the bolt's path. After the warning, the bolt travels at laser_speed px/frame. laser_mode='rapid' in Level 2 means only one laser is alive at a time (new lasers are not spawned while one is active). laser_mode='single' in Levels 1 and 3 allows multiple simultaneous lasers. laser_grace in Level 1 prevents any lasers for the first 120 frames (2 s) of the level.

7.4 SunOrb and BigOrb Behaviour

SunOrbs spawn in pairs using SunOrb.find_safe_pos() which guarantees a minimum separation of 220px between orbs. Both grow over 100 frames with an ease-out cubic curve. During growth, global screen shake ramps from 4 to 22px and zoom_scale can reach 1.04x. Three layers of SRCALPHA fire halos plus _FlameParticle emitters create the visual. BigOrb in Level 3 is functionally identical but has RADIUS_MAX=68 (vs 55), spawns in groups of 3 every 300 frames, and triggers a stronger shake (45px) and zoom (1.12x).

7.5 Black Hole (Level 3)

The black hole is triggered when level_timer <= 1800 (the last 30 seconds). It spawns at screen centre and grows to 220px radius over 3 seconds. Directional gravity is applied to the player each frame:

```
base_force = (0.04 + gravity_scale * 0.28) * direction_factor
player.vel += pull_dir * base_force
```

Figure 4 — Directional gravity: direction_factor ranges 0.6-1.0 based on player velocity relative to hole

All hazards (exploders, lasers, minis, sun orbs, big orbs) are also pulled toward the hole and removed when they enter its event horizon. The collapse phase begins 40 frames before expiry. The kill radius (200px) is slightly smaller than the visual radius (220px) to give a small margin of safety.

7.6 Power-Up System

PowerUpManager controls one power-up on screen at a time. The first power-up spawns at a random moment within the first 10 seconds. After collection, the next spawns after a 15-second cooldown. Uncollected power-ups expire after 5 seconds (with a blinking warning in the final second). The three types are:

- Shield (cyan): Absorbs the next hit entirely. Duration: 5 seconds. Visual: orbiting shield nodes on two concentric rings + bubble shell.
- +HP (green): Absorbs 2 hits (or 10 seconds, whichever first). Visual: green orbit particles.
- Slow-motion (blue): Hazards move at 60% speed. Duration: 7 seconds. Visual: blue ripple rings + screen tint.

8. Audio: Music and Sound Effects

8.1 Audio Assets

File	Channel / Usage	Volume	Description
assets/menu_music/bgmusic.mp3	mixer.music	0.70	Background music on warning/menu screen
assets/levels/the_awakening.mp3	mixer.music	0.65	Level 1 gameplay music (looped)
assets/levels/pulse_overdrive.mp3	mixer.music	0.65	Level 2 gameplay music (looped)
assets/levels/chaos-protocol.mp3	mixer.music	0.65	Level 3 gameplay music (looped)
assets/menu_music/stagemusic.mp3	mixer.music	0.75	Stage-cleared transition music (looped while player waits)
assets/sfx/hit.mp3	Channel 1	0.60	Player hit SFX
assets/sfx/death.mp3	Channel 2	1.00	Player death SFX
assets/sfx/laser.mp3	Channel 3	1.00	Laser bolt fire SFX
assets/sfx/sunorb.mp3	Channel 4	1.00	SunOrb / BigOrb activation SFX (boosted)
assets/sfx/explosionorb.mp3	Channel 5	1.00	Exploder detonation SFX (boosted)
assets/sfx/slowmo.mp3	Channel 6	1.00	Slow-motion power-up pickup SFX
assets/sfx/shield.mp3	Channel 6	1.00	Shield power-up pickup SFX
assets/sfx/extra.mp3	Channel 6	1.00	+HP power-up pickup SFX

8.2 Audio Triggering and Mixing

The mixer is initialised with explicit settings (44100 Hz, -16 bit, stereo, buffer 512) before `pygame.init()` to avoid default device settings. 16 channels are allocated and channel 0 is reserved so music never interrupts SFX. The `_play()` helper routes sounds to their dedicated channels to prevent cutting off when the same sound is triggered on overlapping events.

Channels 4 (big_orb) and 5 (explode) are designated as 'boosted': both the Sound object volume and the channel volume are set to 1.0 each call to counteract pygame's internal mixing attenuation and ensure these impactful sounds are clearly audible during intense gameplay.

8.3 Music Flow

Gameplay music is loaded and looped via `pygame.mixer.music`. On level completion, the gameplay music fades out over 600ms, then `stage_music.mp3` starts. After the player presses C and the countdown completes, `stage_music.mp3` fades over 500ms before the next level's music loads. This ensures clean audio transitions without pops or overlap. The `_fade_out_bg_music()` and `_fade_out_stage_music()` helpers track state flags to prevent duplicate fade calls.

9. Visual and Audio Implementation Details

9.1 Rendering Pipeline

All game elements are drawn to `virtual_screen` (900x600 Surface). At the end of each frame, `virtual_screen` is scaled to the monitor resolution using `pygame.transform.smoothscale` with an optional `zoom_scale` multiplier (normally 1.0, spikes to ~1.1 on impacts). A random `shake_offset` vector is applied to the blit position and decays at 15% per frame (`shake_amount *= 0.85`). This gives screen shake a natural exponential decay rather than a harsh cut.

9.2 Transparency and Glow Effects

All glow, aura, and particle effects use `pygame.Surface` with `pygame.SRCALPHA` to ensure circular/soft shapes do not leave visible rectangular bounding boxes on the background. This is critical for the neon aesthetic — every glow layer is a transparent surface blitted using standard alpha compositing. The game never uses `pygame.Surface.convert()` on glow surfaces, which would strip the alpha channel.

9.3 Particle Systems

Three distinct particle systems operate simultaneously: `HitParticle` (burst on player hit, 14 particles, 20-35 frame lifetime), `NearMissParticle` (8 particles spawned when a laser passes within 36px of the player, 12-22 frame lifetime), and `_FlameParticle` (continuous emission from `SunOrb/BigOrb` edges with colour gradient from yellow-white at birth to deep red at death). All particle lists use list comprehension removal to avoid concurrent modification issues.

9.4 Screen Effects

Chromatic aberration is applied for 14 frames after a player hit by offsetting red and blue channel copies of the virtual surface. The Level 3 intensity flash (`_update_l3_flash`) fires random multi-layer red/white bursts every 80-200 frames, with spontaneous probability of 2.2% per frame. A vignette overlay (pre-baked at two intensities) darkens screen edges — the intense variant activates during black hole events. The scrolling background grid shifts 0.4px per frame and uses level-specific base colours with a sinusoidal beat pulse for visual rhythm.

10. Code Snippets and Pseudocode (Appendix)

10.1 OpenCV → MediaPipe Frame Loop

The background thread loop in `tracking.py` demonstrates the minimal camera-to-landmark pipeline:

```
success, frame = self.cap.read()
frame = cv2.flip(frame, 1)
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
results = self.hands.process(rgb_frame)
if results.multi_hand_landmarks:
    tip = results.multi_hand_landmarks[0].landmark[8]
    screen_x = np.interp(tip.x, [0.15, 0.85], [0, self.width])
```

Figure 5 — Minimal MediaPipe landmark extraction pipeline

10.2 Smoothing and Game-Space Conversion

```
self._raw_pos = Vector2(screen_x, screen_y)
self.index_pos = self.index_pos.lerp(self._raw_pos, 0.55)
```

The lerp factor 0.55 means each frame the cursor moves 55% of the remaining distance to the raw landmark position. This creates a critically-damped-style follow that eliminates jitter without introducing lag.

10.3 Level Configuration to Spawn Logic

The following pseudocode shows how `levels.py` parameters feed directly into the spawn system in `dodge.py`'s main loop:

```
data = LEVELS[level_index]
# Exploder spawn
if level_timer % data['exploder_rate'] == 0:
    for _ in range(data['exploder_count']):
        exploders.append(Exploder())

# Laser spawn with grace period
if laser_cd <= 0:
    elapsed = SURVIVAL_TIME - level_timer
    if elapsed > data['laser_grace']:
        lasers.append(Laser(speed=data['laser_speed'])
```

Figure 6 — Spawn logic driven by `levels.py` configuration values

10.4 Full `levels.py` Content

The complete level configuration file is reproduced below:

```
LEVELS = [
    { 'id': 1, 'name': 'THE AWAKENING', 'music': 'assets/music.mp3',
      'exploder_rate': 240, 'exploder_count': 2, 'laser_rate': 120,
      'laser_count': 1, 'laser_speed': 22, 'laser_mode': 'single',
      'laser_grace': 120, 'has_big_orb': False, 'big_orb_freq': 0,
      'mini_count': 4, 'particle_speed': 3, 'sun_orb_rate': 0 },
    { 'id': 2, 'name': 'PULSE OVERDRIVE', 'music': 'assets/music2.mp3',
      'exploder_rate': 140, 'exploder_count': 3, 'laser_rate': 60,
      'laser_count': 1, 'laser_speed': 34, 'laser_mode': 'rapid',
      'laser_grace': 0, 'has_big_orb': False, 'big_orb_freq': 0,
      'mini_count': 6, 'particle_speed': 6, 'sun_orb_rate': 300 },
    { 'id': 3, 'name': 'CHAOS PROTOCOL', 'music': 'assets/music3.mp3',
      'exploder_rate': 120, 'exploder_count': 5, 'laser_rate': 30,
      'laser_count': 1, 'laser_speed': 28, 'laser_mode': 'single',
```

```
'laser_grace': 0, 'has_big_orb': True, 'big_orb_freq': 999999,
'mini_count': 6, 'particle_speed': 7, 'sun_orb_rate': 240 },
]
```

Figure 7 — Complete levels.py configuration

11. Testing, Performance, and Limitations

11.1 Recommended Camera Settings

- Use a webcam capable of 640x480 at 30 FPS or higher. 60 FPS cameras reduce input lag noticeably.
- Ensure even, frontal lighting — backlighting or side shadows cause MediaPipe detection failures.
- A plain, non-cluttered background improves detection reliability.
- Maintain a distance of 40-80 cm from the camera for optimal landmark accuracy.

11.2 Performance Targets

Metric	Target	Notes
Render FPS	60	Enforced by clock.tick(60). May drop on slow GPUs.
Camera latency	< 50 ms	Achieved by BUFFERSIZE=1 and daemon thread.
Tracking latency	1-2 frames	One camera frame behind due to thread read lag.
Model complexity	0 (lite)	Fastest MediaPipe model. Sufficient for single-hand tracking.
Particle budget	< 300 active	Near-miss particles capped via cooldown; flame particles self-expire.

11.3 Known Edge Cases

- Occlusion: When the hand partially exits the camera frame, detection may drop to the wrist. The lerp smoothing prevents jarring jumps; the cursor freezes at last known position.
- Skin tone variation: MediaPipe Hands is generally robust across skin tones, but very dark environments may require increasing min_detection_confidence.
- Multiple hands: Only the first detected hand is used. If another person's hand enters frame, MediaPipe may switch to their hand, causing control loss.
- Level-jump keys (1/2/3): These bypass the normal intro animation check and can create brief visual artifacts if pressed during transitions.

11.4 QA Checklist

- Verify webcam opens successfully (cap.isOpened() check is implicit — cap.read() failure silently continues the loop).
- Test all three levels from start to completion on target hardware.
- Verify all audio files are present in assets/; missing files are logged with [audio] MISSING but do not crash the game.
- Test death and respawn flow: HP reaches 0, fade to black, level intro replays, gameplay resumes.
- Test stage-cleared transition: timer hits 0, music fades, stage-cleared screen shows, countdown to next level.

- Verify power-up collection and each power-up effect (shield block, extra hits, slow-motion hazards).
- Test black hole trigger in Level 3 at the 30-second mark: hazards should be pulled in and player should feel gravitational force.

12. References and Assets

12.1 Source Files

File	Description
dodge.py	Main game loop, rendering, entity classes, screen effects, audio, menus
tracking.py	HandTracker: camera capture, MediaPipe integration, smoothing, gestures
levels.py	LEVELS list: all per-level configuration parameters
powerups.py	Power-up system: PowerUp, PowerUpManager, player state helpers, HUD drawing

12.2 Asset Files

File Path	Type	Usage
assets/menu_music/bgmusic.mp3	Audio	Menu/warning screen background music
assets/levels/the_awakening.mp3	Audio	Level 1 music
assets/levels/pulse_overdrive.mp3	Audio	Level 2 music
assets/levels/chaos-protocol.mp3	Audio	Level 3 music
assets/menu_music/stagemusic.mp3	Audio	Stage-cleared transition music
assets/sfx/hit.mp3	Audio	Player hit sound
assets/sfx/death.mp3	Audio	Player death sound
assets/sfx/laser.mp3	Audio	Laser fire sound
assets/sfx/sunorb.mp3	Audio	SunOrb / BigOrb activation sound
assets/sfx/explosionorb.mp3	Audio	Exploder detonation sound
assets/sfx/slowmo.mp3	Audio	Slow-motion pickup sound
assets/sfx/shield.mp3	Audio	Shield pickup sound
assets/sfx/extra.mp3	Audio	+HP pickup sound
assets/img/icon.png	Image	Window/taskbar icon (optional)

12.3 Third-Party Libraries

Library	Version (recommended)	Purpose
Python	3.10+	Core language
pygame	2.5+	Game loop, rendering, audio
opencv-python (cv2)	4.8+	Camera capture and image processing
mediapipe	0.10+	Hand landmark detection
numpy	1.24+	Coordinate interpolation (np.interp)

12.4 Credits

- Game design, programming, and all visual implementation: Edrean Supremo
- Music: Geometry Dash soundtracks
- Sound effects: Pixabay
- Hand tracking: Google MediaPipe (Apache 2.0 license)

13. One-Page Summary

NEON DODGE: LEVEL OVERDRIVE — AT A GLANCE

Category	Details
What it is	A 3-level webcam-controlled arcade dodge game built in Python/pygame
How you play	Point your index finger at the camera and move your hand to steer the player avatar
Win condition	Survive 60 seconds per level across all 3 levels
Lose condition	Lose all 3 HP — level restarts automatically
Levels	1: The Awakening (intro pacing) 2: Pulse Overdrive (faster, sun orbs) 3: Chaos Protocol (black hole, big orbs, dense hazards)
Key tech	OpenCV camera → MediaPipe Hands → 55% lerp smooth → pygame physics
Hazards	Exploders, Laser bolts, Sun Orbs, Big Orbs, Black Hole, MiniCircle fragments
Power-ups	Shield (5 s), Slow-motion (7 s), +HP (10 s or 2 hits)
Audio	5 music tracks + 8 SFX; dedicated mixer channels; fade transitions
Visual FX	Screen shake/zoom, chromatic aberration, particle systems, Level 3 flash, vignette, glow layers
Dependencies	Python 3.10+, pygame 2.5+, opencv-python 4.8+, mediapipe 0.10+, numpy 1.24+
Author	Edrean Supremo © 2026 Educational use